mock provider should respond with a status of 200 and the correct header type. The response body is represented as a variable called `EXPECTED_BODY`. Let's declare this variable outside our test:

```
const EXPECTED_BODY = { id: 1, name: "My GraphQL movie", year: 1999 };
```

Feel free to update `EXPECTED_BODY` to be of any value, as long as the data type matches. Using Pact matchers, we're saying that `EXPECTED_BODY` should have an ID, the movie name should be something similar to "My GraphQL movie", and the year should be something similar to the year 1999.

Remember that with contract testing, the consumer should care more about the shape of the data rather than the data itself. We use the `eachLike` matching rule to ensure the type matches all the values. This way, the test should still pass if the provider returns different values, but the types are similar.

7.3.4 Verifying the consumer test and generating the contract

The next step is to call the function the GraphQL consumer created, responsible for fetching all the movies. In this example, the `getMovies` function is called and stored in the `response` variable:

```
test('returns the correct response', async () => {
  const response = await getMovies();
});
```

Once the `getMovies` function is called, we need to verify that the actual data returned by the GraphQL consumer matches the expected data. We do this by comparing the actual data returned and the expected data we defined:

```
test('returns the correct response', async () => {
  const response = await getMovies();
  expect(response.movies[0]).toEqual(EXPECTED_BODY);
});
```

To verify the consumer expectation against Pact and to generate the contract, we need to add the following methods in our test: